Project title: Customer Churn Prediction

This notebook contains the code for a customer churn prediction project for our term project

Subject: APPLIED MATHEMATICAL CONCEPTS FOR MACHINE LEARNING CRN-48756-202502

Dataset: https://www.kaggle.com/datasets/sonalshinde123/customer-churn-prediction-dataset

Github Repo: https://github.com/varsha-jai/MathAI_Term_Project

Authors: Varsha Jaikrishnan, Ali Karimzadeh Bangi, Aparna Mohankumar, Andy Phan, Jordan Palacios, Damir Akhunov, Iva Kovacevic, Lalit Kumar

Date: 31/01/2026

Objective: The goal of this telecom churn ML project is to predict which customers are likely to leave (churn) soon, so the company can intervene before it happens.

Dataset: This dataset is a synthetic customer churn dataset designed to simulate real-world telecom customer behavior. It is generated using business-driven rules based on customer tenure, billing amount, contract type, service usage, and support interactions. Controlled randomness and noise are added to avoid perfect patterns and make the dataset suitable for realistic machine learning classification tasks. The dataset is ideal for beginners to practice exploratory data analysis, feature engineering, and customer churn prediction using machine learning models.

Target variable: Churn

Evaluation metric: Accuracy, Recall and F1-score

In [1]:
```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file
import matplotlib.pyplot as plt # a library for data visualization
import seaborn as sns # great for making informative plots more easily
import time
import sys

from sklearn import preprocessing
from sklearn.preprocessing import StandardScaler # scaling numerical columns
from sklearn.model_selection import train_test_split # splitting data to tra
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.metrics import precision_score, f1_score, recall_score
from sklearn.metrics import confusion_matrix, accuracy_score,classification_
from sklearn.pipeline import Pipeline
from sklearn.metrics import make_scorer, recall_score, classification_report

import warnings
warnings.filterwarnings('ignore')
```

1. Obtain a large classification database and load it. Provide a description of the dataset, including explanation of various features.

In [2]:
```
telecom_churn_data = pd.read_csv("customer_churn_dataset.csv")
```

Our dataset is called "customer_churn_dataset.csv". It has 20,000 rows, which are synthetic customer information. It also has 11 features, for churn prediction.

This dataset provides synthetic customer churn information. It simulates real-world telecom customer behavior. In also includes controlled randomness and noise to avoid perfect patterns and to make it suitable for beginner machine learning task of customer churn prediction using machine learning models.

The dataset is generated using business-driven rules based on features such as customer tenure, billing amount, contract type, etc.

Some of the features are as follows: "tenure" - how long customers have been a customer in months "monthly_charges", their monthly bill "total_charges", the amount they have paid in total "Contract", whether they are month-to-month or on a longer plan "Payment_method", how how they pay, i.e. credit or debit "Internet_service", such as Fiber, etc. "tech_support", whether they have received tech support "Support_calls", the number of times they have called support.

The target feature "churn", which is a Yes/No field, indicates whether they are still with the company or they have left.

This dataset is a mix of numerical and categorical features for churn prediction.

Exploratary Data Analysis

In [3]:
```
telecom_churn_data.head() # A quick overview of the first five rows to under
```

Out[3]:

	customer_id	tenure	monthly_charges	total_charges	contract	payment_method
0	1	52	54.20	2818.40	Month-to-month	Credit
1	2	15	35.28	529.20	Month-to-month	Debit
2	3	72	78.24	5633.28	Month-to-month	Debit
3	4	61	80.24	4894.64	One year	Cash
4	5	21	39.38	826.98	Month-to-month	UPI

In [4]:
```
print(telecom_churn_data.shape)
```

```
(20000, 11)
```

The following line of code tells us that out of the 11 variables, we have 3 integer type, 2 float type and 6 discrete type

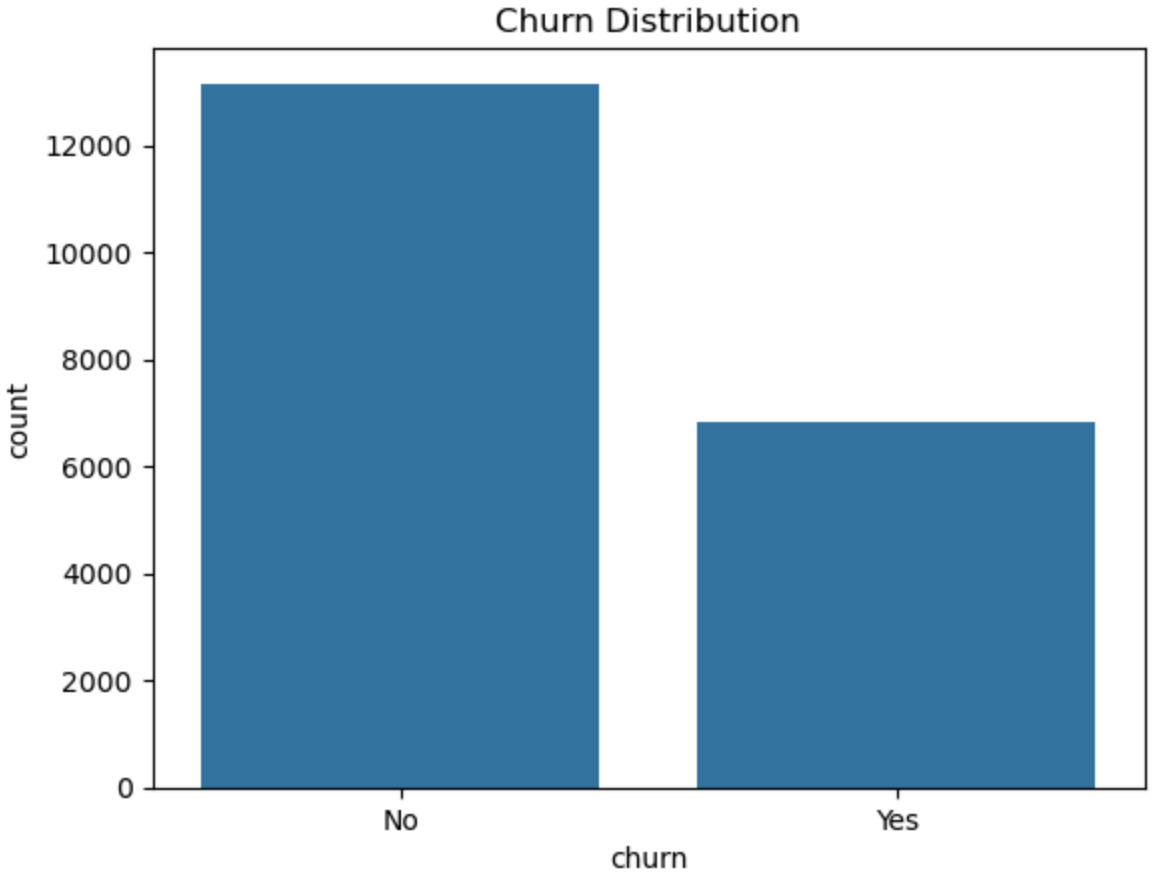
In [5]:
```
print(telecom_churn_data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customer_id       20000 non-null  int64  
 1   tenure            20000 non-null  int64  
 2   monthly_charges   20000 non-null  float64
 3   total_charges     20000 non-null  float64
 4   contract          20000 non-null  object 
 5   payment_method    20000 non-null  object 
 6   internet_service  17987 non-null  object 
 7   tech_support      20000 non-null  object 
 8   online_security   20000 non-null  object 
 9   support_calls     20000 non-null  int64  
 10  churn             20000 non-null  object 
dtypes: float64(2), int64(3), object(6)
memory usage: 1.7+ MB
None
```

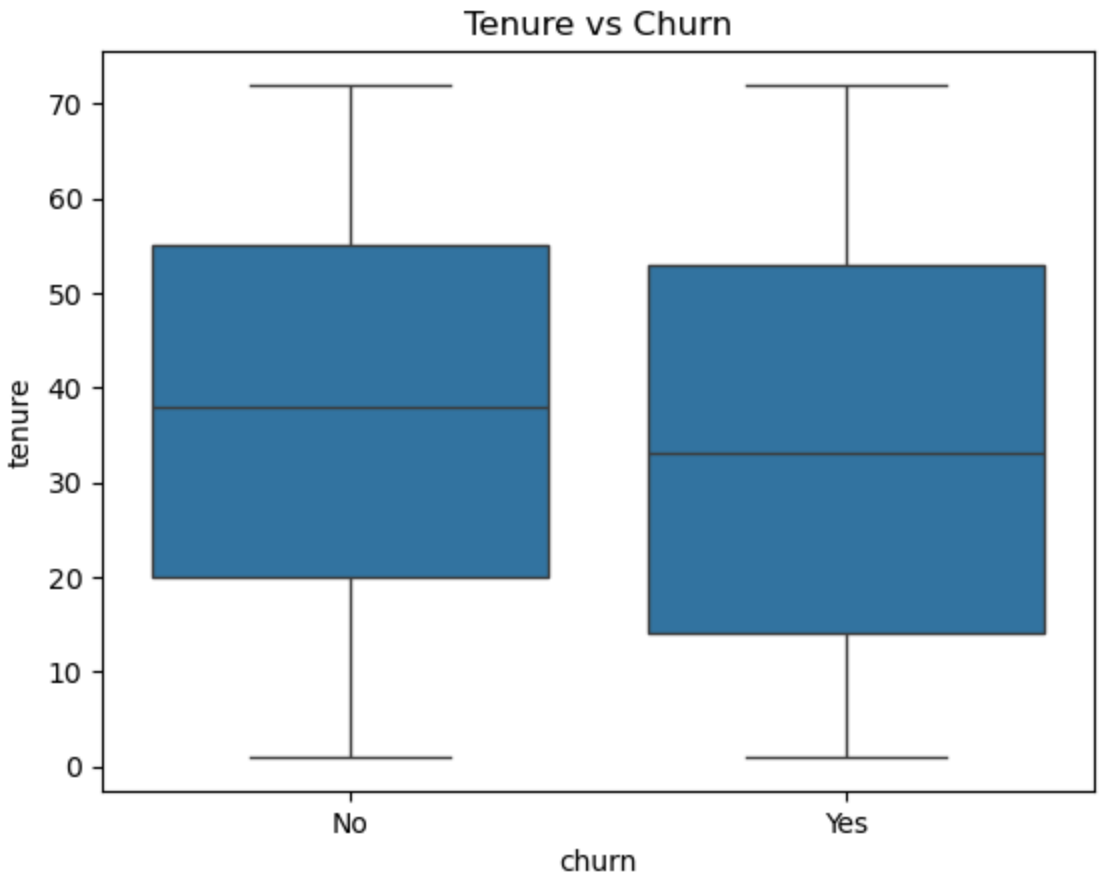
In [6]:
```
y = telecom_churn_data['churn']
print(f'Percentage of Churn:  {round(y.value_counts(normalize=True)[1]*100,2
print(f'Percentage Non_Churn: {round(y.value_counts(normalize=True)[0]*100,2
```

```
Percentage of Churn:  34.22 %  --> (6843 customers)
Percentage Non_Churn: 65.79  %  --> (13157 customers)
```

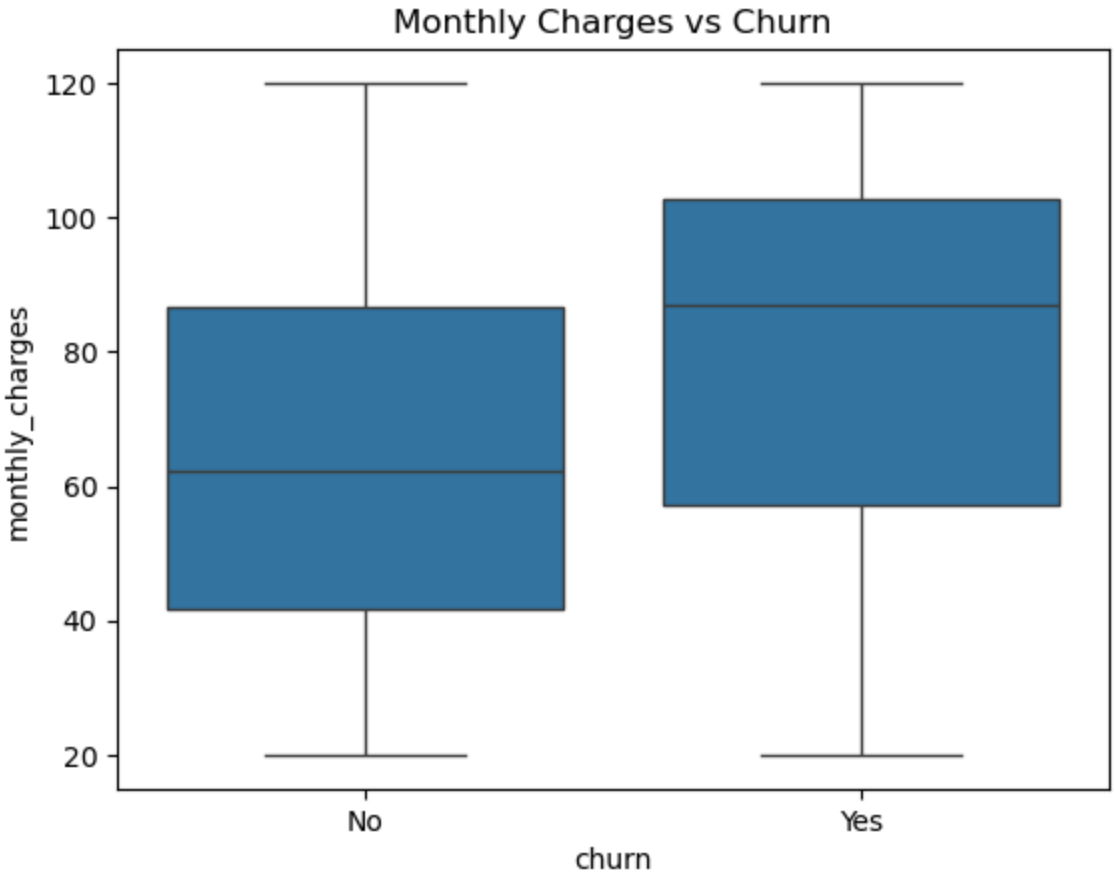
In [7]:
```
#### Visualizing churn distribution to check for imbalance

plt.figure()
sns.countplot(x="churn", data=telecom_churn_data) # Showing the distributior
plt.title("Churn Distribution")
plt.show()
```



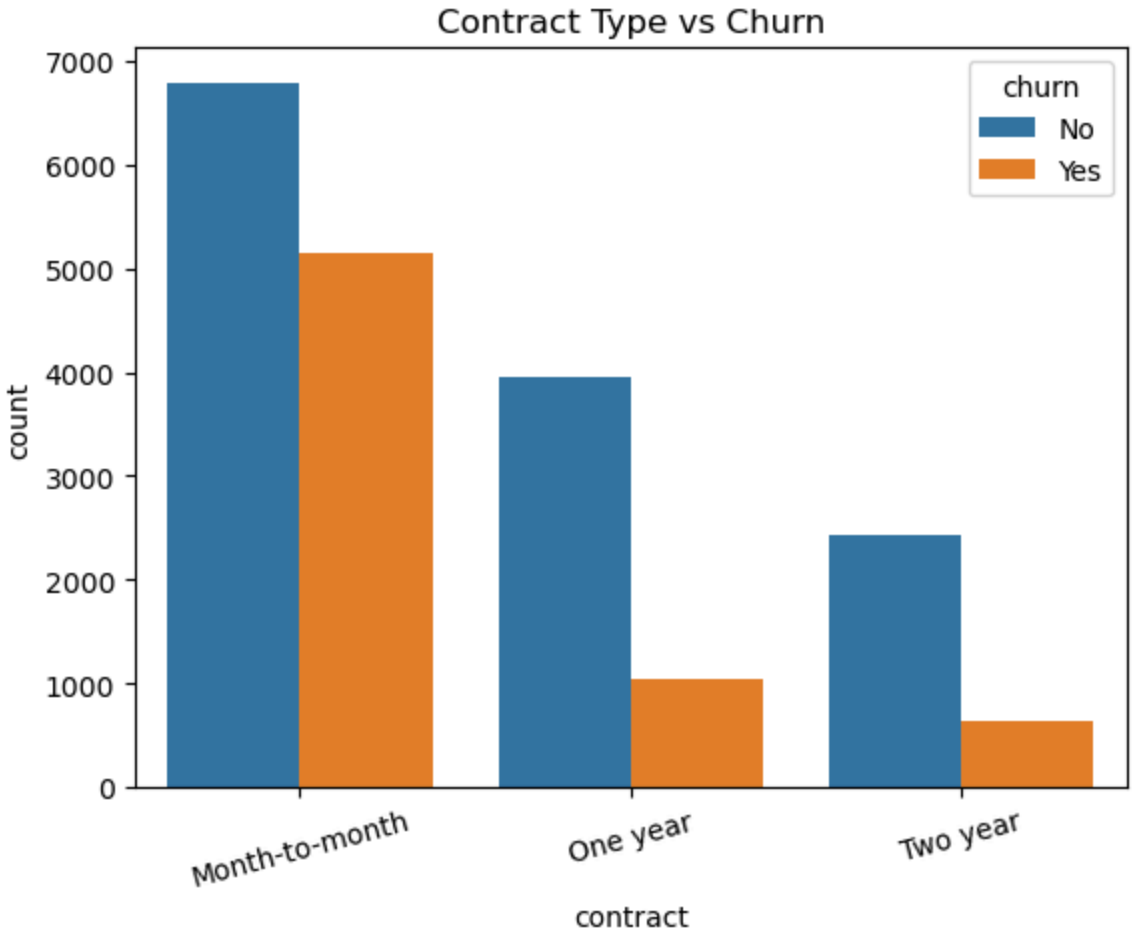
In [8]:
```
plt.figure()
sns.boxplot(x="churn", y="tenure", data=telecom_churn_data) # helping us spo
plt.title("Tenure vs Churn")
plt.show()

# It seems that tenured customers more likely to stay and not churn
```



In [9]:
```
plt.figure()
sns.boxplot(x="churn", y="monthly_charges", data=telecom_churn_data) # showi
plt.title("Monthly Charges vs Churn")
plt.show()

# It seems that customers with low monthly charges are more likely to stay a
```



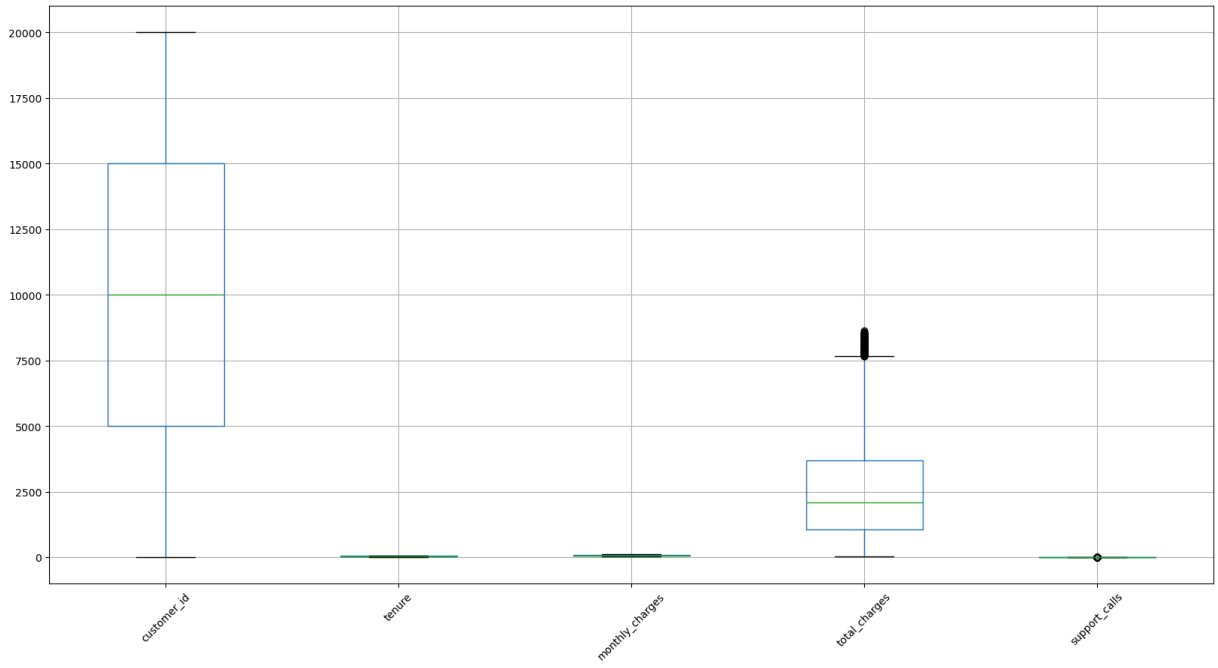
In [10]:
```
plt.figure()
sns.countplot(x="contract", hue="churn", data=telecom_churn_data) # Comparir
plt.title("Contract Type vs Churn")
plt.xticks(rotation=15)
plt.show()

# It seems that customers with longer contract are more likely to stay and r
```



Checking for outliers

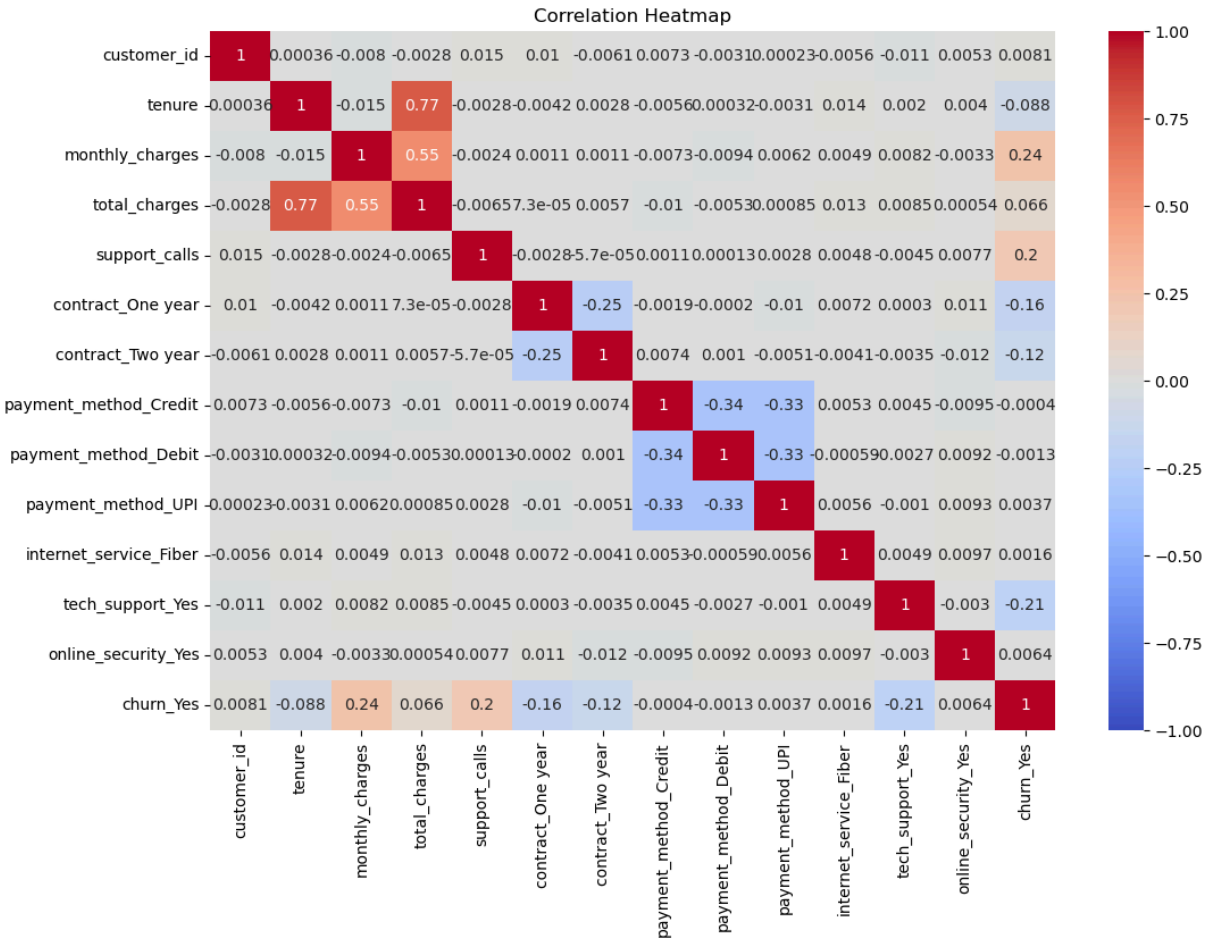
In [11]:
```
plt.figure(figsize=(20,10))
telecom_churn_data.boxplot()
plt.xticks(rotation=45)
```

Out[11]:
```
(array([1, 2, 3, 4, 5]),
 [Text(1, 0, 'customer_id'),
  Text(2, 0, 'tenure'),
  Text(3, 0, 'monthly_charges'),
  Text(4, 0, 'total_charges'),
  Text(5, 0, 'support_calls')])
```



Checking correlation with a heatmap

In [12]:
```
encoded = pd.get_dummies(telecom_churn_data, drop_first=True)
corr = encoded.corr()

# Plot heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.show()
```



Feature Selection

In [13]:
```
#Dropping customer_id based on the notion that it does not affect our models

telecom_churn_data = telecom_churn_data.drop(columns=['customer_id']) # It i
print(telecom_churn_data.shape)
```

```
(20000, 10)
```

Pre-processing the data

Dealing with Missing data

In [14]:
```
telecom_churn_data.isnull().sum() # checking for missing values in every col
```

Out[14]:
```
tenure                0
monthly_charges       0
total_charges         0
contract              0
payment_method        0
internet_service    2013
tech_support          0
online_security       0
support_calls         0
churn                 0
dtype: int64
```

In [15]:
```
# The "internet_service" has 2013 missing values, which is a moderate amount

mode_value = telecom_churn_data['internet_service'].mode()[0] # To avoid dro
print(mode_value) # mode() gives us the value that occurs most frequently in

telecom_churn_data['internet_service'] = telecom_churn_data['internet_servic
telecom_churn_data.isnull().sum() # making sure that there is no more missir
```

```
Fiber
```

Out[15]:
```
tenure              0
monthly_charges     0
total_charges       0
contract            0
payment_method      0
internet_service    0
tech_support        0
online_security     0
support_calls       0
churn               0
dtype: int64
```

Converting categorical features into numerical

In [16]:
```
# Here in this code bloack we make a list of all the categorical features in

colnames = []
for x in telecom_churn_data.columns[:-1]:
    if telecom_churn_data[x].dtype == 'object':
        colnames.append(x)
colnames
```

Out[16]:
```
['contract',
 'payment_method',
 'internet_service',
 'tech_support',
 'online_security']
```

In [17]:
```
telecom_churn_data = pd.get_dummies(
    telecom_churn_data,
    columns=colnames,
    drop_first=True
)
```

Splitting Data into train and test

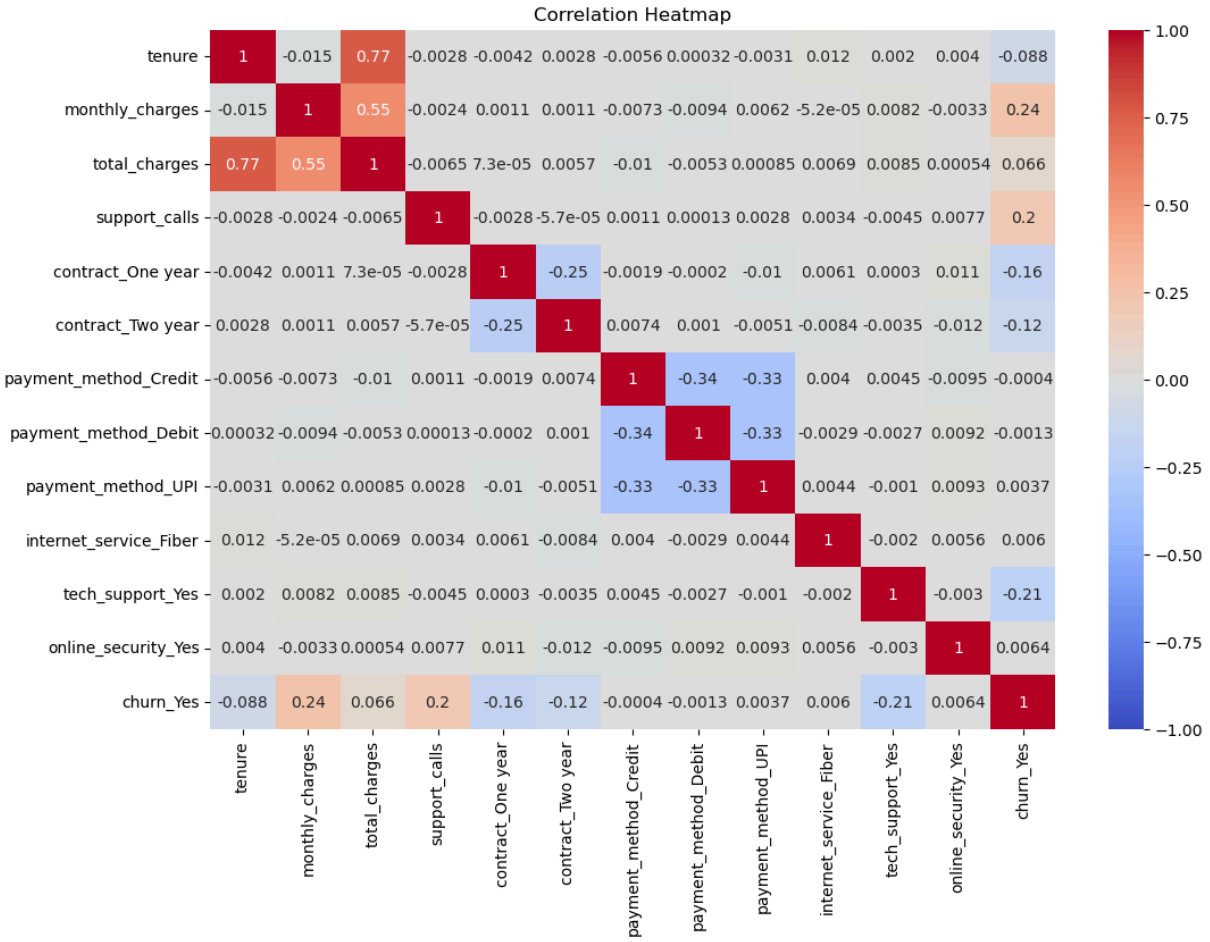
In [18]:
```
# Target
y = telecom_churn_data["churn"]

# Convert target to 0/1
y = y.replace({False: 0, True: 1, "No": 0, "Yes": 1}).astype(int)

# Features
X = telecom_churn_data.drop(columns=["churn"])

# Sanity check: X must have no object columns
obj_cols = X.select_dtypes(include=["object"]).columns
print("Object columns still in X:", list(obj_cols))

# If any object columns remain, one-hot encode them
if len(obj_cols) > 0:
    X = pd.get_dummies(X, columns=obj_cols, drop_first=True)


#### Spilitting the dataset into train (80%), validation (10%) and test (20%

# 80% train, 20% temp (val + test)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.2, rar

# Split temp evenly: 10% val, 10% test
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.

print("Shapes:", X_train.shape, X_val.shape, X_test.shape)
print("y_train unique:", y_train.unique())

#Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled   = scaler.transform(X_val)
X_test_scaled  = scaler.transform(X_test)
```

```
Object columns still in X: []
Shapes: (16000, 12) (2000, 12) (2000, 12)
y_train unique: [0 1]
```

Using the heatmap after pre-processing

In [19]:
```
encoded = pd.get_dummies(telecom_churn_data, drop_first=True)
corr = encoded.corr()

# Plot heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
```

```
plt.title("Correlation Heatmap")
plt.show()
```



Training the models

4. Use following approaches for classification of the dataset:

A. Logistic Regression

B. Decision Tree

C. Random Forest

D. SGD

E. SVM

In [20]:
```
#Creating lists to gather metrics

#train metrics:
tr_acc, tr_rec, tr_prec = ([] for _ in range(3))
```

```
#validation metrics:
acc, rec, prec, time_taken = ([] for _ in range(4))
```

Base Logistic Regression Model

In [21]:
```
model_logistic = LogisticRegression(max_iter=1000, random_state=45) # Initia
start = time.time()
model_logistic.fit(X_train, y_train) # Training the Logistic Regression on c
end = time.time()

# Predicting
y_train_pred = model_logistic.predict(X_train)
y_val_pred = model_logistic.predict(X_val)
```

In [22]:
```
# Accuracy
tr_accuracy = round(accuracy_score(y_train, y_train_pred)*100,1)
accuracy = round(accuracy_score(y_val, y_val_pred)*100,1)

# Train Report
train_report = classification_report(y_train, y_train_pred)
train_report_1 = classification_report(y_train, y_train_pred, output_dict=Tr

# Validation Report
val_report = classification_report(y_val, y_val_pred)
val_report_1 = classification_report(y_val, y_val_pred, output_dict=True)

# Printing results
print(f"Train Accuracy: {tr_accuracy}")
print(f"Validation Accuracy: {accuracy}")

#Recall
tr_recall_yes  = round(train_report_1['1']['recall'] * 100, 1)
val_recall_yes = round(val_report_1['1']['recall'] * 100, 1)

#Precision
tr_prec_yes  = round(train_report_1['1']['precision'] * 100, 1)
val_prec_yes = round(val_report_1['1']['precision'] * 100, 1)

#Time taken
fit_time = end - start
print(f"Fit time: {fit_time:.2f} seconds")
```

```
Train Accuracy: 78.0
Validation Accuracy: 77.0
Fit time: 0.48 seconds
```

Our Logistic Regretion model successfully identified 54% of the actual churned customers.

In [23]:
```
#Gathering results from log reg
tr_acc.append(tr_accuracy)
tr_rec.append(tr_recall_yes)
tr_prec.append(tr_prec_yes)
```

```
#validation metrics:
acc.append(accuracy)
rec.append(val_recall_yes)
prec.append(tr_prec_yes)
time_taken.append(fit_time)
```

In [24]:
```
print(tr_acc)
print(tr_rec)
print(tr_prec)

print(acc)
print(rec)
print(prec)
print(time_taken)
```

```
[78.0]
[54.3]
[74.6]
[77.0]
[51.7]
[74.6]
[0.4769289493560791]
```

Decision Tree

In [25]:
```
model_DecisionTree = DecisionTreeClassifier(random_state=45,min_samples_leaf
start = time.time()
model_DecisionTree.fit(X_train,y_train)
end = time.time()

# Predicting
y_train_pred = model_DecisionTree.predict(X_train)
y_val_pred = model_DecisionTree.predict(X_val)
```

In [26]:
```
# Accuracy
tr_accuracy = round(accuracy_score(y_train, y_train_pred)*100,1)
accuracy = round(accuracy_score(y_val, y_val_pred)*100,1)

# Train Report
train_report = classification_report(y_train, y_train_pred)
train_report_1 = classification_report(y_train, y_train_pred, output_dict=Tr

# Validation Report
val_report = classification_report(y_val, y_val_pred)
val_report_1 = classification_report(y_val, y_val_pred, output_dict=True)

# Printing results
print(f"Train Accuracy: {tr_accuracy}")
print(f"Validation Accuracy: {accuracy}")


#Recall
tr_recall_yes  = round(train_report_1['1']['recall'] * 100, 1)
val_recall_yes = round(val_report_1['1']['recall'] * 100, 1)
```

```
#Precision
tr_prec_yes  = round(train_report_1['1']['precision'] * 100, 1)
val_prec_yes = round(val_report_1['1']['precision'] * 100, 1)

#Time taken
fit_time = end - start
print(f"Fit time: {fit_time:.2f} seconds")
```

```
Train Accuracy: 84.4
Validation Accuracy: 84.0
Fit time: 0.03 seconds
```

Our Decision Tree model successfully identified 69% of the actual churned customers.

In [27]:
```
#Gathering results from dt
tr_acc.append(tr_accuracy)
tr_rec.append(tr_recall_yes)
tr_prec.append(tr_prec_yes)

#validation metrics:
acc.append(accuracy)
rec.append(val_recall_yes)
prec.append(tr_prec_yes)
time_taken.append(fit_time)
```

In [28]:
```
print(tr_acc)
print(tr_rec)
print(tr_prec)


print(acc)
print(rec)
print(prec)
print(time_taken)
```

```
[78.0, 84.4]
[54.3, 67.4]
[74.6, 83.6]
[77.0, 84.0]
[51.7, 65.4]
[74.6, 83.6]
[0.4769289493560791, 0.026723146438598633]
```

4.C Random Forest

In [29]:
```
model_RandomForest=RandomForestClassifier(n_estimators=25,min_samples_leaf=5
start = time.time()
model_RandomForest.fit(X_train,y_train)
end = time.time()

# Predicting
y_train_pred = model_RandomForest.predict(X_train)
y_val_pred = model_RandomForest.predict(X_val)
```

In [30]:
```
# Accuracy
tr_accuracy = round(accuracy_score(y_train, y_train_pred)*100,1)
accuracy = round(accuracy_score(y_val, y_val_pred)*100,1)

# Train Report
train_report = classification_report(y_train, y_train_pred)
train_report_1 = classification_report(y_train, y_train_pred, output_dict=Tr

# Validation Report
val_report = classification_report(y_val, y_val_pred)
val_report_1 = classification_report(y_val, y_val_pred, output_dict=True)

# Printing results
print(f"Train Accuracy: {tr_accuracy}")
print(f"Validation Accuracy: {accuracy}")


#Recall
tr_recall_yes  = round(train_report_1['1']['recall'] * 100, 1)
val_recall_yes = round(val_report_1['1']['recall'] * 100, 1)

#Precision
tr_prec_yes  = round(train_report_1['1']['precision'] * 100, 1)
val_prec_yes = round(val_report_1['1']['precision'] * 100, 1)

#Time taken
fit_time = end - start
print(f"Fit time: {fit_time:.2f} seconds")
```

```
Train Accuracy: 82.5
Validation Accuracy: 82.0
Fit time: 0.12 seconds
```

Our Random Forest model successfully identified 69% of the actual churned customers.

In [31]:
```
#Gathering results from rf
tr_acc.append(tr_accuracy)
tr_rec.append(tr_recall_yes)
tr_prec.append(tr_prec_yes)

#validation metrics:
acc.append(accuracy)
rec.append(val_recall_yes)
prec.append(tr_prec_yes)
time_taken.append(fit_time)
```

In [32]:
```
print(tr_acc)
print(tr_rec)
print(tr_prec)

print(acc)
print(rec)
print(prec)
print(time_taken)
```

```
[78.0, 84.4, 82.5]
[54.3, 67.4, 59.0]
[74.6, 83.6, 85.3]
[77.0, 84.0, 82.0]
[51.7, 65.4, 56.4]
[74.6, 83.6, 85.3]
[0.4769289493560791, 0.026723146438598633, 0.1228790283203125]
```

4.D SGD

In [33]:
```
model_SGD = SGDClassifier(random_state=45)                                    #creat
start = time.time()
model_SGD.fit(X_train, y_train)
end = time.time()

# Predicting
y_train_pred = model_SGD.predict(X_train)
y_val_pred = model_SGD.predict(X_val)
```

In [34]:
```
# Accuracy
tr_accuracy = round(accuracy_score(y_train, y_train_pred)*100,1)
accuracy = round(accuracy_score(y_val, y_val_pred)*100,1)

# Train Report
train_report = classification_report(y_train, y_train_pred)
train_report_1 = classification_report(y_train, y_train_pred, output_dict=Tr

# Validation Report
val_report = classification_report(y_val, y_val_pred)
val_report_1 = classification_report(y_val, y_val_pred, output_dict=True)

# Printing results
print(f"Train Accuracy: {tr_accuracy}")
print(f"Validation Accuracy: {accuracy}")
```

```
#Recall
tr_recall_yes  = round(train_report_1['1']['recall'] * 100, 1)
val_recall_yes = round(val_report_1['1']['recall'] * 100, 1)

#Precision
tr_prec_yes  = round(train_report_1['1']['precision'] * 100, 1)
val_prec_yes = round(val_report_1['1']['precision'] * 100, 1)

#Time taken
fit_time = end - start
print(f"Fit time: {fit_time:.2f} seconds")
```

```
Train Accuracy: 67.4
Validation Accuracy: 66.5
Fit time: 0.08 seconds
```

In [35]:
```
#Gathering results from rf
tr_acc.append(tr_accuracy)
tr_rec.append(tr_recall_yes)
tr_prec.append(tr_prec_yes)

#validation metrics:
acc.append(accuracy)
rec.append(val_recall_yes)
prec.append(tr_prec_yes)
time_taken.append(fit_time)
```

In [36]:
```
print(tr_acc)
print(tr_rec)
print(tr_prec)

print(acc)
print(rec)
print(prec)
print(time_taken)
```

```
[78.0, 84.4, 82.5, 67.4]
[54.3, 67.4, 59.0, 6.3]
[74.6, 83.6, 85.3, 78.9]
[77.0, 84.0, 82.0, 66.5]
[51.7, 65.4, 56.4, 4.2]
[74.6, 83.6, 85.3, 78.9]
[0.4769289493560791, 0.026723146438598633, 0.1228790283203125, 0.07832479476
928711]
```

4.E SVM

In [37]:
```
from sklearn.svm import SVC
from sklearn.metrics import classification_report

model_SVM = SVC(kernel="rbf", C=1, class_weight="balanced", probability=True
start = time.time()
model_SVM.fit(X_train_scaled, y_train)
end = time.time()
```

```
# Predicting
y_train_pred = model_SVM.predict(X_train_scaled)
y_val_pred = model_SVM.predict(X_val_scaled)
```

In [38]:
```
# Accuracy
tr_accuracy = round(accuracy_score(y_train, y_train_pred)*100,1)
accuracy = round(accuracy_score(y_val, y_val_pred)*100,1)

# Train Report
train_report = classification_report(y_train, y_train_pred)
train_report_1 = classification_report(y_train, y_train_pred, output_dict=Tr

# Validation Report
val_report = classification_report(y_val, y_val_pred)
val_report_1 = classification_report(y_val, y_val_pred, output_dict=True)

# Printing results
print(f"Train Accuracy: {tr_accuracy}")
print(f"Validation Accuracy: {accuracy}")

#Recall
tr_recall_yes  = round(train_report_1['1']['recall'] * 100, 1)
val_recall_yes = round(val_report_1['1']['recall'] * 100, 1)

#Precision
tr_prec_yes  = round(train_report_1['1']['precision'] * 100, 1)
val_prec_yes = round(val_report_1['1']['precision'] * 100, 1)

#Time taken
fit_time = end - start
print(f"Fit time: {fit_time:.2f} seconds")
print(val_report)
```

```
Train Accuracy: 80.5
Validation Accuracy: 79.2
Fit time: 19.01 seconds
              precision    recall  f1-score   support

           0       0.83      0.86      0.84      1315
           1       0.71      0.66      0.68       685

    accuracy                           0.79      2000
   macro avg       0.77      0.76      0.76      2000
weighted avg       0.79      0.79      0.79      2000
```

In [39]:
```
#Gathering results from svm
tr_acc.append(tr_accuracy)
tr_rec.append(tr_recall_yes)
tr_prec.append(tr_prec_yes)

#validation metrics:
acc.append(accuracy)
rec.append(val_recall_yes)
prec.append(tr_prec_yes)
time_taken.append(fit_time)
```

In [40]:
```
print(tr_acc)
print(tr_rec)
print(tr_prec)

print(acc)
print(rec)
print(prec)
print(time_taken)
```

```
[78.0, 84.4, 82.5, 67.4, 80.5]
[54.3, 67.4, 59.0, 6.3, 68.9]
[74.6, 83.6, 85.3, 78.9, 72.7]
[77.0, 84.0, 82.0, 66.5, 79.2]
[51.7, 65.4, 56.4, 4.2, 66.0]
[74.6, 83.6, 85.3, 78.9, 72.7]
[0.4769289493560791, 0.026723146438598633, 0.1228790283203125, 0.07832479476
928711, 19.008352041244507]
```

In [41]:
```
model_types = ["Logistic Regression", "Decision Tree", "Random Forest", "SGD

df = pd.DataFrame({
    "model_type": model_types,
    "Val accuracy": acc,
    "Train accuracy": tr_acc,
    "Val Recall": rec,
    "Train Recall": tr_rec,
    "Val Precision": prec,
    "Train Precision": tr_prec,
    "Val Time_taken": time_taken
})

df
```

Out[41]:

	model_type	Val accuracy	Train accuracy	Val Recall	Train Recall	Val Precision	Train Precision	Val Time_taken
0	Logistic Regression	77.0	78.0	51.7	54.3	74.6	74.6	0.476929
1	Decision Tree	84.0	84.4	65.4	67.4	83.6	83.6	0.026723
2	Random Forest	82.0	82.5	56.4	59.0	85.3	85.3	0.122879
3	SGD	66.5	67.4	4.2	6.3	78.9	78.9	0.078325
4	SVM	79.2	80.5	66.0	68.9	72.7	72.7	19.008352

Parameter tuning using GridSearchCV

In this section, we performed hyperparameter tuning using GridSearchCV in order to identify the best model parameters and compare their cross-validated scores. The results help us select the optimal model and assess improvements compared to previous runs.

GridSearchCV

In [42]:
```
import sys
from sklearn.metrics import make_scorer, recall_score, classification_report

print("Starting GridSearch...")
sys.stdout.flush()

recall_scorer = make_scorer(recall_score, pos_label=1)

# Logistic Regression
log_reg_params = {'C': [0.01, 0.1, 1, 10, 100], 'penalty': ['l1', 'l2'], 'sc
log_reg = LogisticRegression(max_iter=1000, class_weight="balanced")
log_search = GridSearchCV(log_reg, log_reg_params, cv=3, scoring=recall_scor

# Decision Tree
tree_params = {'max_depth': [3, 5, 10, 20, 30, None], 'min_samples_leaf': [1
tree = DecisionTreeClassifier(class_weight="balanced", random_state=42)
tree_search = GridSearchCV(tree, tree_params, cv=3, scoring=recall_scorer, v

# Random Forest
forest_params = {'n_estimators': [50, 100, 200], 'max_depth': [None, 5, 10,
forest = RandomForestClassifier(class_weight="balanced", random_state=42)
forest_search = GridSearchCV(forest, forest_params, cv=3, scoring=recall_scc

# SGD
sgd_params = {'alpha': [0.0001, 0.001, 0.01]}
sgd = SGDClassifier(random_state=42, class_weight="balanced")
sgd_search = GridSearchCV(sgd, sgd_params, cv=3, scoring=recall_scorer, verb
```

```
# SVM Pipeline
svm_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(class_weight="balanced"))
])

svm_params = [
    {"svc__kernel": ["linear"], "svc__C": [0.1, 1, 10, 100]},
    {"svc__kernel": ["rbf"], "svc__C": [0.1, 1, 10, 100], "svc__gamma": ["sc
]
svm_search = GridSearchCV(svm_pipe, svm_params, cv=3, scoring=recall_scorer,

# Fit with progress prints
log_search.fit(X_train, y_train); print("✅ log done"); sys.stdout.flush()
tree_search.fit(X_train, y_train); print("✅ tree done"); sys.stdout.flush()
forest_search.fit(X_train, y_train); print("✅ forest done"); sys.stdout.flu
sgd_search.fit(X_train, y_train); print("✅ sgd done"); sys.stdout.flush()
svm_search.fit(X_train, y_train); print("✅ svm done"); sys.stdout.flush()

print("\nBest params + CV recall:")
print("Logistic Regression:", log_search.best_params_, log_search.best_score
print("Decision Tree:", tree_search.best_params_, tree_search.best_score_)
print("Random Forest:", forest_search.best_params_, forest_search.best_score
print("SGD:", sgd_search.best_params_, sgd_search.best_score_)
print("SVM:", svm_search.best_params_, svm_search.best_score_)

best_log = log_search.best_estimator_
best_tree = tree_search.best_estimator_
best_forest = forest_search.best_estimator_
best_sgd = sgd_search.best_estimator_
best_svm = svm_search.best_estimator_

print("\nSVM Test report:\n", classification_report(y_test, best_svm.predict
```

```
Starting GridSearch...
Fitting 3 folds for each of 10 candidates, totalling 30 fits
✅ log done
Fitting 3 folds for each of 18 candidates, totalling 54 fits
✅ tree done
Fitting 3 folds for each of 24 candidates, totalling 72 fits
✅ forest done
Fitting 3 folds for each of 3 candidates, totalling 9 fits
✅ sgd done
Fitting 3 folds for each of 20 candidates, totalling 60 fits
✅ svm done

Best params + CV recall:
Logistic Regression: {'C': 0.01, 'penalty': 'l1', 'solver': 'liblinear'} 0.7
409591043819596
Decision Tree: {'max_depth': None, 'min_samples_leaf': 5} 0.7161131138348154
Random Forest: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 50}
0.7005855964111191
SGD: {'alpha': 0.01} 0.7382028558840023
SVM: {'svc__C': 0.1, 'svc__gamma': 0.001, 'svc__kernel': 'rbf'} 0.7347473564
047103

SVM Test report:
              precision    recall  f1-score   support

           0       0.83      0.70      0.76      1316
           1       0.55      0.72      0.63       684

    accuracy                           0.71      2000
   macro avg       0.69      0.71      0.69      2000
weighted avg       0.73      0.71      0.71      2000
```

Observations

Logistic Regression: baseline was 51%, while tuned is 74%

Decision Tree: baseline was 65%, while tuned is 71%

Random Forest: baseline was 56%, while tuned is 70%

SGD Classifier: baseline was 42%, while tuned is 73%

SVM: baseline was 66%, while tuned is 73%

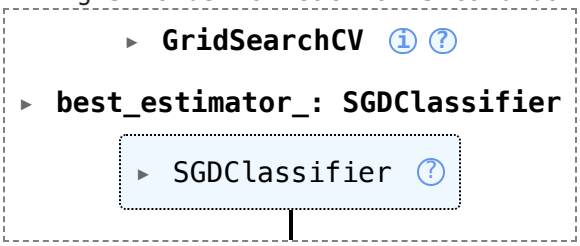
Feature selection modifications

We hypothesized that the payment method might have less direct impact. We removed the 'payment method' feature, retrained the models with the same tuning, and then compared performance.

In [43]:
```
X_train_reduced = X_train.drop(columns=['payment_method'], errors='ignore')
X_test_reduced = X_test.drop(columns=['payment_method'], errors='ignore')

log_search.fit(X_train_reduced, y_train)
tree_search.fit(X_train_reduced, y_train)
forest_search.fit(X_train_reduced, y_train)
sgd_search.fit(X_train_reduced, y_train)
#svm_search.fit(X_train_reduced, y_train)
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Fitting 3 folds for each of 18 candidates, totalling 54 fits
Fitting 3 folds for each of 24 candidates, totalling 72 fits
Fitting 3 folds for each of 3 candidates, totalling 9 fits
```

Out[43]:



In [44]:
```
y_pred_log = log_search.predict(X_test_reduced)
log_recall = recall_score(y_test, y_pred_log, pos_label=1)
print("Logistic Regression Recall:", log_recall)

y_pred_tree = tree_search.predict(X_test_reduced)
tree_recall = recall_score(y_test, y_pred_tree, pos_label=1)
print("Decision Tree Recall:", tree_recall)

y_pred_forest = forest_search.predict(X_test_reduced)
forest_recall = recall_score(y_test, y_pred_forest, pos_label=1)
print("Random Forest Recall:", forest_recall)

y_pred_sgd = sgd_search.predict(X_test_reduced)
sgd_recall = recall_score(y_test, y_pred_sgd, pos_label=1)
print("SGD Classifier Recall:", sgd_recall)

y_pred_svm = svm_search.predict(X_test_reduced)
svm_recall = recall_score(y_test, y_pred_svm, pos_label=1)
print("SVM Recall:", svm_recall)
```

```
Logistic Regression Recall: 0.7324561403508771
Decision Tree Recall: 0.7119883040935673
Random Forest Recall: 0.6652046783625731
SGD Classifier Recall: 0.06725146198830409
SVM Recall: 0.7207602339181286
```

In [ ]: